🍽️ AI-Powered Restaurant Feedback System - User Dashboard

📋 Project Overview

This is the **User Dashboard** component of an AI-powered feedback management system. Customers can submit ratings and reviews, and receive instant AI-generated responses. The system uses Google's Gemini AI to analyze feedback, generate personalized responses, and provide actionable insights for restaurant management.

Key Features

✅ **Interactive Star Rating** (1-5 stars)
✅ **Text Review Input** with validation
✅ **AI-Generated Responses** using Gemini API
✅ **Smart Feedback Analysis** (summary + recommendations)
✅ **Dual Storage System** (Supabase + CSV fallback)
✅ **Recent Feedback Display** (sorted positive to negative)
✅ **Beautiful UI** with custom color scheme
✅ **Production-Ready Database** with Supabase

🎨 Design

The dashboard features a professional color scheme:

- **Primary Accent**: Sage Green (#6B8E23) - Buttons, headers
- **Secondary**: Deep Charcoal (#36454F) - Text
- **Ratings**: Vibrant Gold (#FFC72C) - Star ratings
- **Background**: Crisp Cream (#FAF9F6) - Main background
- **Accents**: Light Tan (#E7E5DF) - Borders, dividers

🚀 Quick Start

Prerequisites

- Python 3.9 or higher
- Google Gemini API key (free tier available)
- Supabase account (optional but recommended)

Installation

1. **Clone the repository**

```bash
git clone <your-repo-url>
cd task2-user-dashboard
```

2. **Install dependencies**

```bash
pip install -r requirements.txt
```

3. **Set up Gemini API**

```bash
# Copy the example file
cp .env.example .env

# Edit .env and add your Gemini API key
# Get your key from: https://makersuite.google.com/app/apikey
```

4. **Set up Supabase (Recommended)**
   - Follow the complete guide: SUPABASE_SETUP.md
   - Add your Supabase credentials to `.env`
   - The app works with CSV only if Supabase is not configured

5. **Run the application**

```bash
streamlit run app.py
```

The dashboard will open in your browser at `http://localhost:8501`

🔑 Getting Your Gemini API Key

1. Visit Google AI Studio
2. Sign in with your Google account
3. Click "Create API Key"
4. Copy your key and add it to `.env`:

```
GEMINI_API_KEY=your_actual_key_here
```

Note: The free tier includes:

- 60 requests per minute
- 1,500 requests per day
- Perfect for this project!

📁 Project Structure

```
task2-user-dashboard/
│
├── app.py                # Main Streamlit application
├── llm_client.py         # Gemini API integration
├── prompts.py            # Optimized prompt templates
├── utils.py              # Helper functions (dual storage)
├── supabase_client.py    # Supabase database client
├── requirements.txt      # Python dependencies
├── .env.example          # Environment variables template
├── .gitignore            # Git ignore rules
├── README.md             # This file
├── SUPABASE_SETUP.md     # Complete Supabase setup guide
│
└── feedback_data.csv     # CSV backup (auto-created)
```

🔧 How It Works

User Flow

1. **Select Rating**: User clicks on 1-5 stars
2. **Write Review**: User types their feedback
3. **Submit**: User clicks "Submit Feedback"
4. **AI Processing**: System calls Gemini API with optimized prompts
5. **Response Generation**: AI generates:
    - Personalized user response
    - Internal summary
    - Recommended actions
6. **Display**: User sees AI response instantly
7. **Storage**: All data saved to Supabase (and CSV backup)

Data Storage

Primary Storage: Supabase (PostgreSQL)

Table: `feedback`

Column	Type	Description
`id`	BIGSERIAL	Auto-incrementing primary key
`timestamp`	TIMESTAMPTZ	ISO format timestamp
`rating`	INTEGER	1-5 star rating (validated)
`review`	TEXT	User's review text
`ai_response`	TEXT	AI-generated user message
`summary`	TEXT	AI-generated summary
`recommended_actions`	TEXT[]	Array of action items
`created_at`	TIMESTAMPTZ	Row creation timestamp

Backup Storage: CSV

- `feedback_data.csv` - Same structure as above
- Automatically used if Supabase unavailable

- Perfect for local development

Setup Guide: See SUPABASE_SETUP.md

🤖 Prompt Engineering

The system uses carefully optimized prompts:

System Prompt

- Sets context (restaurant feedback assistant)
- Defines JSON output structure
- Specifies tone guidelines for different ratings
- Ensures structured, parseable responses

User Prompt

- Rating-specific instructions (5-star vs 1-star)
- Includes actual review text
- Guides tone and content
- Ensures actionable outputs

Example Output:

```json
{
  "user_response": "Thank you for your wonderful feedback! We're thrilled you enjoyed the food...",
  "summary": "Customer praised food quality but mentioned slow service during peak hours.",
  "recommended_actions": [
    "Increase staffing during dinner rush",
    "Implement table management system",
    "Provide server training on efficiency"
  ]
}
```

🎯 Key Features Explained

1. Smart Rating Analysis

The AI adapts its response based on rating:

- **5 stars**: Enthusiastic gratitude
- **4 stars**: Warm thanks with improvement notes
- **3 stars**: Balanced acknowledgment
- **2 stars**: Sincere apology with action steps
- **1 star**: Deep apology with urgent interventions

2. Recent Feedback Display

- Shows last 5 submissions
- **Sorted positive to negative** (5-star → 1-star)
- Color-coded rating badges
- Timestamps ("2 min ago", "1 hour ago")

3. Error Handling

- Fallback responses if API fails
- Input validation (rating required, min review length)
- Graceful degradation with predefined templates

🚢 Deployment to Streamlit Cloud

Step-by-Step Guide

1. **Push to GitHub**

```bash
git init
git add .
git commit -m "Initial commit - User Dashboard"
git remote add origin <your-github-repo>
git push -u origin main
```

2. **Deploy on Streamlit Cloud**
    - Go to share.streamlit.io
    - Sign in with GitHub
    - Click "New app"
    - Select your repository
    - Set main file: `app.py`
    - Click "Deploy"
3. **Add Secrets**
    - In your Streamlit Cloud dashboard
    - Go to your app settings
    - Add to "Secrets":

```toml
GEMINI_API_KEY = "your_actual_key_here"
```

4. **Your app is live!** 🎉

🧪 Testing

Test the Gemini Connection

```bash
python llm_client.py
```

Expected output:

```
✅ Gemini API connection successful!
Testing feedback analysis...
✅ Analysis successful!
--- User Response ---
Thank you for your feedback! ...
```

Test Individual Modules

```bash
# Test prompts
python prompts.py

# Test utilities
python utils.py
```

📊 Usage Examples

Example 1: Positive Review (5 stars)

Input:

- Rating: ⭐⭐⭐⭐⭐
- Review: "Amazing food and excellent service! Best restaurant in town."

AI Response:

> "Thank you so much for your wonderful 5-star review! We're absolutely thrilled to hear you loved both the food and service. Your support means the world to our team, and we can't wait to welcome you back soon!"

Example 2: Mixed Review (3 stars)

Input:

- Rating: ⭐⭐⭐
- Review: "Food was great but we waited 40 minutes for a table."

AI Response:

> "Thank you for sharing your honest feedback. We're glad you enjoyed the food, and we sincerely apologize for the long wait time. We're working on improving our seating process to ensure a smoother experience on your next visit."

Example 3: Negative Review (1 star)

Input:

- Rating: ⭐
- Review: "Terrible experience. Food was cold and service was rude."

AI Response:

> "We are truly sorry to hear about your disappointing experience. This is not the standard we set for ourselves, and we take full responsibility. We're taking immediate action to address the issues you've raised regarding both food quality and service. Please reach out to us directly so we can make this right."

🔍 Troubleshooting

Issue: "GEMINI_API_KEY not found"

Solution: Make sure `.env` file exists and contains your API key

Issue: "JSON parsing error"

Solution: The system automatically falls back to predefined templates. Check your API key validity.

Issue: CSV file not updating

Solution: Check file permissions. The CSV is created automatically on first run.

Issue: Star rating not showing

Solution: Clear browser cache and refresh the page

📈 Next Steps (Task 2 Part 2)

After completing the User Dashboard, you'll build the **Admin Dashboard** which will:

✅ Read from the same `feedback_data.csv`
✅ Display all submissions in a table
✅ Show analytics (rating distribution, trends)
✅ Filter by rating, date, keywords
✅ Export data to different formats

🤝 Contributing

This is an assignment project, but feedback is welcome!

📝 License

This project is part of the Fynd AI Intern Take-Home Assessment.

📧 Support

For questions about this project:

- Check the Gemini API Docs
- Review the Streamlit Docs

✨ Credits

- **AI Model**: Google Gemini 1.5 Flash
- **Framework**: Streamlit
- **Design**: Custom UI with sage green & gold theme

Built with ❤️ for the Fynd AI Intern Assessment

🚀 **Ready to deploy!** Follow the deployment guide above.